The Hong Kong Polytechnic University
Department of Computing

COMP4913 Capstone Project
Final Reprot

Tool for learning a programming language

Student Name: LI KA KI

Student ID No.: 24027673D

Programme-Stream Code: 61435-SYC

Supervisor: Dr. Ken Yiu

Co-examiner: Dr. Jason Zhang

2nd Assessor: Dr. Max Pei

Submission Date: 10 April-2025

Abstract

In the current era of artificial intelligence, programming skills have become an essential ability for university students, especially as they support emerging technical areas like machine learning, large language models, and data-driven information systems. Python is commonly considered an appropriate introductory programming language due to its readable features and robust ecosystem. However, many learners face busy schedules that is to say, they find it challenging to dedicate substantial, continuous study periods. Meanwhile, popular web-based learning solutions often fail to provide an ideal starting experience for beginners, owing to reasons such as technical error messages, text-heavy learning content, limited practice features, and insufficiently clear difficulty progression.This progress report describes the development approach and partial implementation of a mobile-based "Python Learning App" supporting short, flexible study sessions. The proposed solution highlights a structured, level-based learning path, various practice formats - for instance multiple-choice questions, fill-in-the-blanks, coding challenges, and mini-project style assignments - simplified actionable error explanations, and cross-session progress tracking feature. Early results indicate that the main learning sequence and code execution process function properly. Future work will concentrate on usability testing activities, enhancing execution reliability and safety features, and incorporating an AI chatbot tutor for real-time student assistance, to put it simply.

Table of contents

1. Background

In the current AI-driven era, programming has become an essential skill for university students, not only for computer science majors but also for learners in many other disciplines. As AI-related fields continue to expand,such as machine learning, large language models (LLMs), and big data—students increasingly want to learn programming languages and build real applications that connect directly to these fast-growing areas [1].

Among many programming languages, Python is widely recognized as one of the most important languages for AI development. Its impact is especially strong in machine learning, deep learning, and data science because it is easy to learn, has an extensive ecosystem, and benefits from strong community support [2]. This popularity is also supported by real-world software development data. In a large-scale mining study of 31,066 machine-learning-related projects on GitHub, Python was identified as the most common language used in ML projects, with many projects depending on major Python libraries such as TensorFlow and scikit-learn [3]. Beyond academic settings, Python is also widely applied across industries—including FinTech, transportation, travel, and healthcare—showing that learning Python can provide practical value and career relevance [4].

However, despite the importance of learning programming, many university students face a major challenge: time. Students often have busy daily schedules filled with classes, assignments, part-time jobs, and extracurricular activities. As a result, it can be difficult for them to commit to long and consistent study sessions needed to build programming skills. Although many online learning platforms exist, a large portion of them are designed primarily for desktop web use, and the learning experience can feel less engaging for beginners, In contrast, Garzón et al [5] said that mobile learning has emerged as a strategy with multiple benefits to promote quality education, mobil learning platforms can take advantage of interactive design patterns and short learning sessions to support learning on the go; mobile learning also can break the traditional limitations of time and location, providing more flexible learning opportunities [6].

To address these issues, this project proposes developing a mobile application that enables students to learn programming during commuting time or short free-time periods throughout the day. The key goal is to provide an engaging and beginner-friendly learning experience that is optimized for mobile usage. Instead of relying on long reading-based lessons, the app will emphasize interactive learning features such as short concept explanations, guided exercises, quick quizzes, progress tracking, and motivational elements . By making learning more accessible and more enjoyable in small time blocks,Microlearning leverages familiar, modern learning technology in short bursts to motivate and stimulate learners while they are on the go [7], the application aims to help students maintain consistent practice, strengthen foundational programming concepts, and improve learning efficiency over time.

2. Related Work

Platform	Level-based Path	Practice Types	Feedback & Error Explanation	Progress Tracking	Mobile Learning Fit	Chatbot Support
W3Schools	Limited	Mostly reading + simple examples	Technical errors; limited beginner explanation	Limited	Web-first	No
Tutorials Point	Limited	MC + coding challenges	Moderate feedback	Limited	Web-first	No
Mimo (Mobile App)	No	Short exercises + guided tasks	Beginner-friendly	Yes	Very good	No
Lrarn Python EmbarkX (Mobile App)	Yes	MC + fullin the blank	Moderate feedback	Yes	Very good	No
Our App	**Yes**	**MC + fill-blank + coding + mini projects**	**Simplified 1–2 sentence explanations**	**Progress + wrong-book**	**Mobile-first, bite-sized**	**Chatbot Support + hints**

Table 1: Compare exist platform functionality

Overall, W3Schools and TutorialsPoint organize learning mainly by topic categories instead of using a structured level-based path. That is to say, learners can freely jump between topics. On the other hand, Mimo follows a more step-by-step flow where users usually start from beginner content, which may reduce flexibility, especially for learners who want to target specific weak areas. To put it simply, Learn Python (EmbarkX) offers clearer choices about levels and lets users switch between levels or topics, making it easier to match learning content with current ability. Regarding mobile learning, W3Schools and TutorialsPoint are largely designed for desktops and are less convenient on phones, while Mimo is more phone-friendly but limits free usage through a daily attempts system, which may break continuous practice for some learners.

In terms of practice, W3Schools and TutorialsPoint mainly rely on multiple-choice questions, resulting in limited variety. Mimo and Learn Python (EmbarkX) provide multiple-choice questions and also fill-in-the-blank exercises, but mini-project style practice is generally missing across the platforms, which may reduce opportunities for hands-on coding and applying knowledge in real situations. Feedback quality also differs: W3Schools often provides weak or not immediate feedback and its error explanations can be too complex for beginners. TutorialsPoint offers moderate

feedback, while Mimo and Learn Python (EmbarkX) tend to give more helpful guidance for newcomers, for instance by offering hints or showing the correct answer after wrong attempts.

For keeping track of progress, W3Schools and TutorialsPoint mainly show overall completion percentages and are not consistently tracking lesson-by-lesson progress to put it simply (Table 1), while Mimo and Learn Python (EmbarkX) offer a clearer way of seeing how far they've gotten through lessons. Finally, none of the platforms we looked at provides an AI helper system for real-time questions and answers. This points to a gap that is important for new learners who often find themselves stuck on ideas or mistakes and need quick help right in the moment. Findings indicate that AI helper improved students' knowledge gains and programming abilities, particularly in writing readable and logically sound code [8].Therefore, the project we are working on aims to fill this gap by bringing together structured learning by levels, richer ways to practice, simpler explanations for errors, and that AI helper system to support learners during practice.

3. Problem Definition and Objective

Based on our earlier look at existing Python learning places and what we found out during the project stage, several important problems came up. These issues could make learning less effective and lower learner enthusiasm, especially for beginners who learn mostly on their phones during short periods of time.

3.1. Beginner-unfriendly error messages

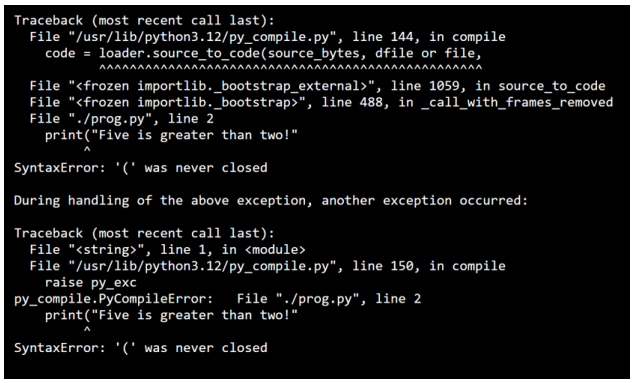


Figure 3.1 Example of a complex/technical error message shown to beginners on W3Schools. Source: Screenshot captured by the author(captured on 2025-11-30).

A big problem for new learners is the trouble they have understanding error messages. In W3Schools, the error output tends to be quite technical and is not presented in a way that makes it easy for beginners. This makes it difficult for learners to figure out what went wrong or how to correct their code (Figure 3.1). To be more specific, the error message is often long, uses technical terms, and it does not really tell beginners what they should do next in a straightforward way. Error messages are an important tool programmers use to help find and fix mistakes or issues in their code. When an error message is unhelpful, it can be difficult to find the issue and may impose additional challenges in learning the language and concepts [9].

To solve this problem, the app provides more straightforward and actionable error explanations that are more beginner-friendly. for frequent beginner mistakes, such as syntax errors, the system gives a simple explanation along with clear suggestions about how to fix that particular issue.

3.2. Limited practice formats

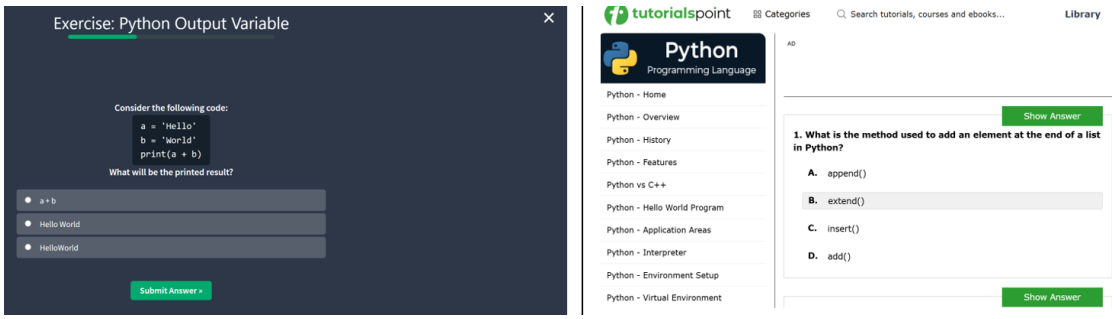


Figure 3.2 Example showing limited practice formats (mainly MCQs) on W3Schools/TutorialsPoint.
Source: Screenshot captured by the author(captured on 2025-11-30).

Research on student participation shows that monotonous learning activities can lead to disengagement in online learning environments [10]. That is to say, effective programming learning really benefits from varied hands-on practice. However, our inspection indicates that platforms such as W3Schools and TutorialsPoint provide limited practice formats, with exercises mostly restricted to multiple-choice questions. This situation may reduce chances for learners to practice coding and debugging in realistic scenarios, which is important for skill development (Figure 3.2).

To address this issue , the app will introduce multiple practice types to enhance hands-on learning. the app must include two or three different exercise formats like MCQ, fill-in-the-blank, and coding tasks or small project exercises.

3.3. Lack of structured level progression

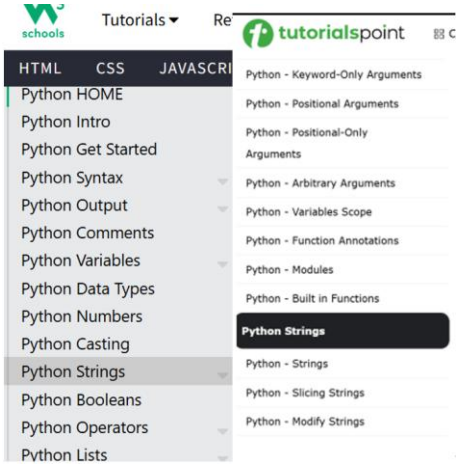


Figure 3.3 Example of a topic-based learning structure without a clear level progression
Source: Screenshot captured by the author(captured on 2025-11-30).

Research in online learning has found that when courses don't have clear progressions in difficulty and proper gradations, students find them either too difficult or too boring, leading to increased frustration and dropout rates [11]. Therefore, a level-based progression system, for instance from beginner to intermediate and then advanced, helps learners study systematically while choosing appropriate content. Our review suggests that some platforms don't provide a structured level path, and learning content is mainly organized by topics rather than difficulty levels. This can make it harder for beginners to follow a coherent progression or identify suitable starting points, to put it simply. An example of this topic-based learning structure without clear progression is shown (Figure 3.3).

To solve this problem, the app will create a level-based structure, such as a learning track from beginner through intermediate to advanced, so learners follow a proper sequence. Users can pick a level for example and access lessons and exercises arranged in a clear step-by-step manner.

3.4. Insufficient immediate learning support

When learners encounter difficulties, they often need help immediately, such as hints, explanations or answering questions in real time. However, the platforms reviewed generally lack such real-time support features, meaning that there's no AI chatbot tutor available. This can make learners feel frustrated more easily and less likely to keep trying when they face errors or conceptual challenges. To put it simply, Kleij et al [12]. state that both meta-analysis research and experiments show that feedback which is timely and explains the reasoning, that is to say interpretive feedback, can greatly improve learning outcomes. Conversely, if feedback comes late or isn't provided, this weakens learning effectiveness, especially for learners who started with less knowledge .

To address insufficient immediate support, the app will offer timely feedback and guidance during practice, including hints or explainers after incorrect tries. That is to say, timely and helpful feedback can boost learning outcomes, particularly for those with little prior knowledge. After any wrong answer or submission, the app returns immediate feedback plus at least one helpful hint or explanation.

3.5. Low engagement due to text-heavy learning and perceived steep learning curve

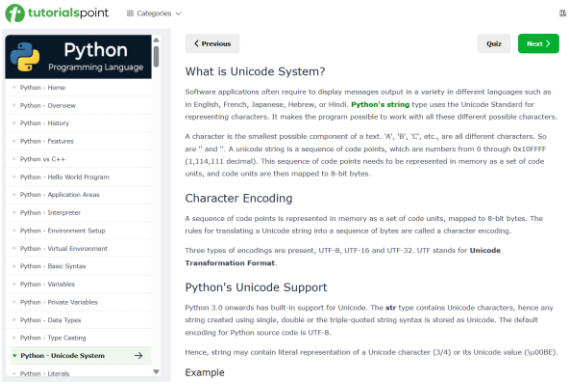


Figure 3.5 Example of a text-heavy explanation
Source: Screenshot captured by the author(captured on 2025-11-30).

Another problem identified is that some platforms rely too much on text-based explanations(Fig 3.5). This approach can seem monotonous and too demanding for beginners. Because learning materials are presented in long text sections mostly with very limited chances for interaction, learners might feel the learning curve is steep and thus become more likely to stop trying. Research in programming education indicate that beginner is already experiencing a high cognitive load when learning programming, therefore, instructional design should reduce unnecessary burdens [13].This issue is particularly relevant when learning happens on mobile devices, where people generally prefer shorter, more interactive content and information. Research shows that programming learning sees high failure and quitting rates, and one major reason for this involves learners viewing the content as too complex and overwhelming. This perception reduces their motivation and makes them less persistent, meaning they don't stick with it as long [14].

To improve engagement for mobile learning and lower drop-out risks (Section 3.5), the app will use short, interactive units suitable for mobile contexts, which can help reduce overwhelming text and perceived high difficulty challenges.Each lesson is designed as a short module with various interactive elements, for instance, a brief explanation plus a quick practice session, and the app includes progress indicators to put it simply, like showing completion status, to encourage users to keep learning moving forward

4. Problem–Solution

4.1: The length of the error messages is too high and is technical.

It has excessive error message length and is technical in nature. The general challenge with learning Python is that default error messages can be long, technical, and confusing to a beginner user. The standard traceback has more rich debugging details, but a novice might be lost trying to figure out what is the most critical part of the message and what is wrong. In order to correct this issue, the project intended to devolop with a completely tailor-made error-message module. The idea behind this module was to convert technical Python error messages into easier to understand and less technical feedback to enable the learners to understand errors faster and enjoy their coding experience,Its implementation is described in Section 6.3.

4.2: Inconsiderable amount of variety of practice questions.

The system does not just offer a single exercise type to give it extra content and meaning. Multiple-choice and coding exercises are presented in the present prototype. In Cloud Firestore exercise content is stored and is loaded by the app according to the lesson chosen. To run the code exercises, users can type Python code in the app and submit it to the backend to be executed then the output/error is sent back and displayed right away as feedback,Its implementation is described in Section 6.2.

4.3: Unstructured materials and materials that have no levels of learning.

In order to address the issue of unstructured learning material we sorted the material and categorised it into distinct levels (e.g., beginner, intermediate, and advanced) and taught the lessons in a sequential order. Every lesson and exercise is marked with a level and order index in Firestore in order to enable the app to present them in a structured learning course. The app has completion status in the progress data of the user, and progress is guided by simply rules (e.g., one must complete the previous lesson before going to the next one), and can still be able to study the previous material,Its implementation is described in Section 6.4.

4.4: Insufficient immediate learning support.

Existing learning platforms generally do not provide real-time assistance when learners encounter difficulties during practice. When beginners face Python errors or do not understand a concept, they may need immediate explanations, hints, or follow-up guidance. Without such support, learners can become frustrated and lose motivation to continue.

To address this issue, the proposed system integrates an AI chatbot as an immediate learning support tool. The chatbot is designed to answer Python-related questions in real time, explain concepts in simple language, and provide brief guidance when learners get stuck. In addition, prompt engineering is applied to keep the chatbot responses focused, concise, and suitable for beginner learners. This implementation is described in Section 6.1.

4.5: Low user engagement due to extensive text content and a steep learning curve.

To address low engagement, The application applies short and interactive learning units better fitted in mobile devices to close the low engagement. Lessons are given in a summary format with the criterion of exercises, as opposed to the textual explanation of each lesson in detail. The design is beneficial as it prevents cognitive load; therefore beginners find it simpler to understand the learning material.

Moreover, the application offers immediate outcome and an indicator of progress on completion of each activity. The system helps in keeping the learners motivated by showing them their progress. This helps in making the learning process stimulating and less stressing and more appropriate in short courses of learning. Details of its implementation can be found in Section 6.5.

5. Design

5.1 System Designs

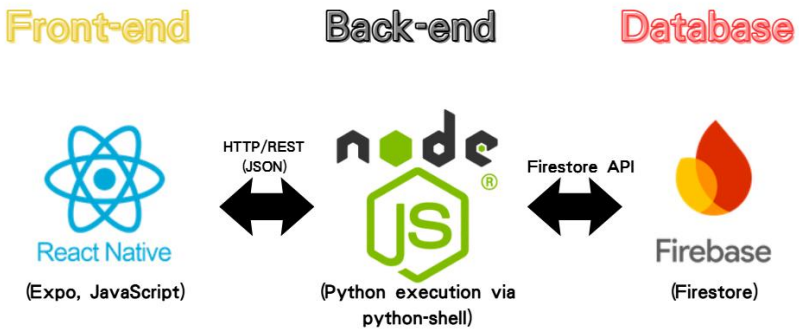


Figure 5.1 Overall system architecture of the proposed Python learning application.

The system is developing three key components, including a react native mobile application, Node.js backend API server, and Firebase (Figure 5.1). The user interface, i.e. lessons, exercises, the Python input area are handled by the mobile app, and the orchestration of the python code is achieved by invoking the python-shell to run the Python code followed by the output or error message being sent back to the mobile app, and the result is shown. user-related data is stored in firebase i.e. the learning progress and history of the exercises. By doing so, the application remains light thus allowing independent code execution and data storage.

5.2 UI Design

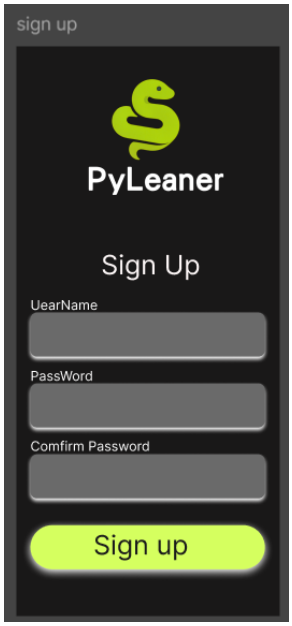


Figure 5.2 UI design of sign up page

Figure 5.3 UI design of login in page

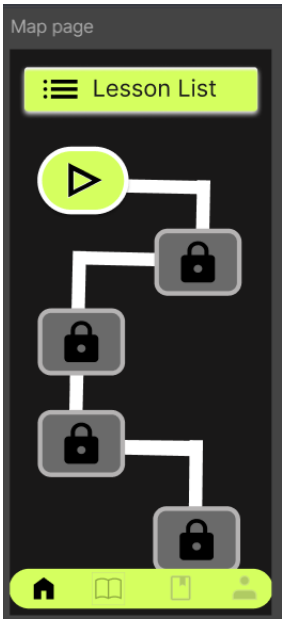


Figure 5.4 UI design of Home page

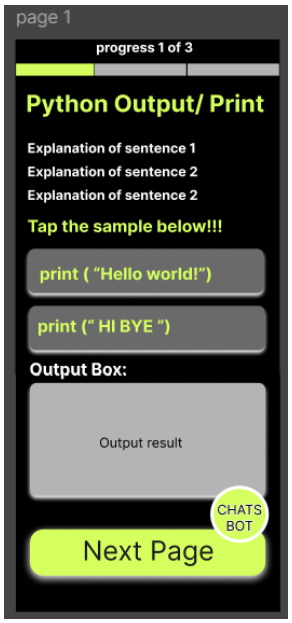


Figure 5.5 UI design of lesson page

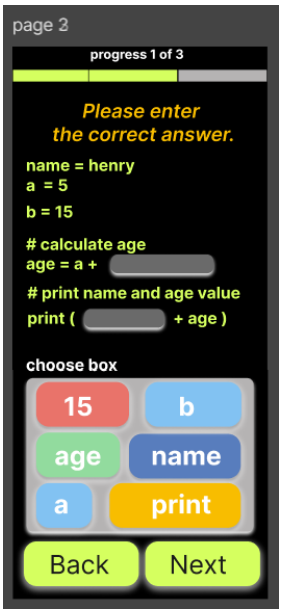


Figure 5.6 UI design of lesson page

Figure 5.7 UI design of chat bot page

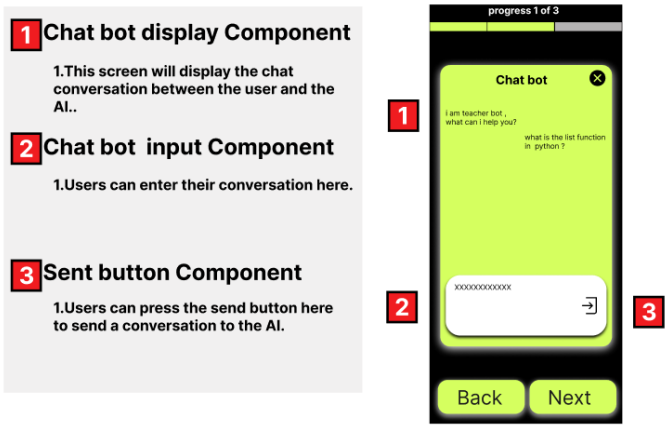


Figure 5.8 Componet describe of chat bot page

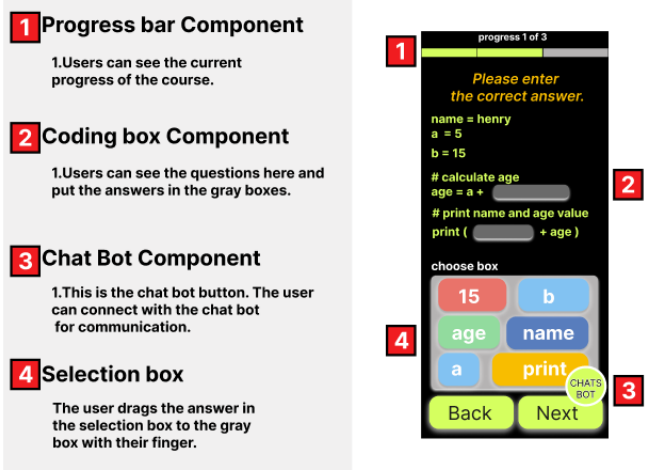


Figure 5.9 Componet describe of lesson page

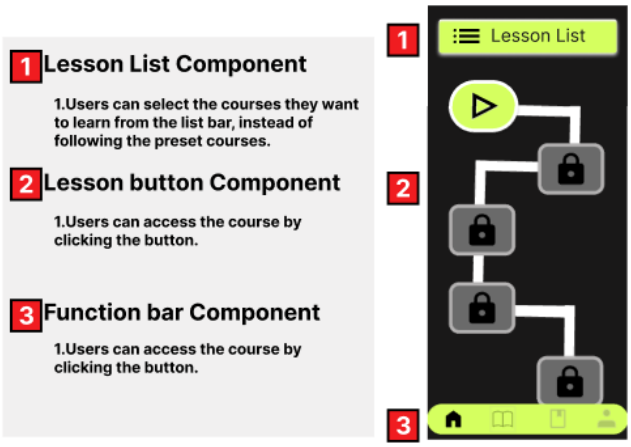


Figure 5.10 Componet describe of Home page

Figure 5.11 Ui design of color allocation

As far as the actual design of the UI is concerned, our potential consumers are mostly university students, a young age group that is not afraid of technology products. That is why, the general visual manner is centered on technological, transparent, and productive. In terms of the color scheme, the color used as the main accent is a glowing, fluorescent green color in combination with a black/dark gray background to provide excellent contrast and, consequently, improve readability, as well as simplify the process of recognizing key objects (main buttons, level/progress indicators, interactive prompts, etc.). Fluorescent colors may be reduced and used to highlight essential processes and indicators to eliminate visual fatigue; in general terms of the text and paragraphs, they are written in white or light grays to facilitate reading even when working long hours.

Regarding the structure design, we take a simple information display on the form of a card, and, as a modular representation of everything, we have Lesson, Practice, Feedback, Progress functions, where one can find their next activity in the short bursts of free time. It ensures consistency in the workflow (e.g. a fixed place to return to, a single style of button, and the semantics of icons) in order to make the learning curve as light as possible. And last but not the least, regarding the feedback design, the system includes an easily readable visual hierarchy of the presentation of the results of the execution: the achievements are represented in a sensible hierarchy by successful passes, error messages, and prompts, thus helping beginners not to feel overwhelming with the abundance of technical data which would make them more willing to stick with the system and carry out exercises.

5.3 Database design

Figure 5.12 NoSQL database design of structure

The database that we have adopted in our project is Cloud Firestore due to its ability to store both teaching content information and the learners information that is related to learning. The lesson materials are kept as global collections in order to allow the app to load dynamically and refresh it without re-compiling the client. Moreover, under the users documents, there is user specific information such as learning progress, history

of exercise attempt, among others. This design keeps shared content distinctly apart and the user records so that there is greater maintainability and scalability. A document is defined by its Firestore document ID, and relationships are defined with reference fields (e.g. lessonId and exerciseId) rather than SQL-style foreign keys. The Firebase Authentication and Firestore security rules are applied to make sure that users can read and write only their learning data but the content of lessons are read-only by the clients.

6. Implementation

6.1 AI Chatbot Implementation

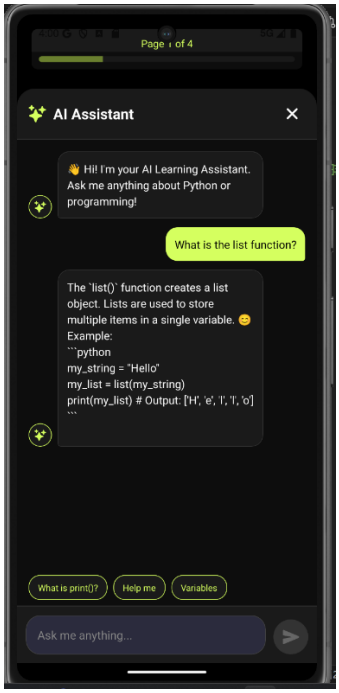


Figure 6.1.1 Chatbot response with prompt engineering

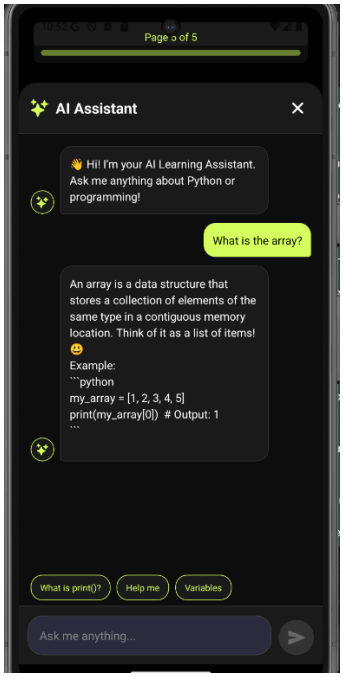


Figure 6.1.2 Chatbot response python question

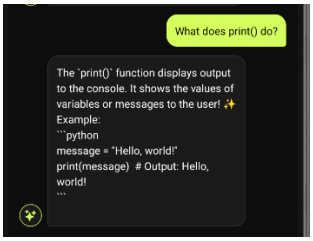


Figure 6.1.3 Chatbot response python question

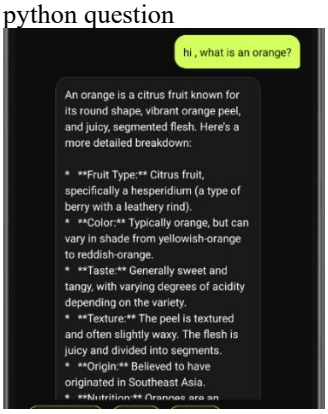


Figure 6.1.4 Chatbot response without prompt constraint

```
const SYSTEM_PROMPT = `You are a friendly and helpful Python programming tutor for beginners.
Your role is to:
- Explain Python concepts in simple, easy-to-understand terms
- Provide clear code examples when helpful
- Encourage learners and celebrate their progress
- Answer questions about Python syntax, functions, loops, variables, etc.
- Keep responses concise but informative
- Use emojis to make responses engaging
- If asked about topics outside Python/programming, politely redirect to programming topics
```

Figure 6.1.5 chatbot prompt command

The software also has an artificial intelligence-powered chatbot that will offer learners on demand assistance when they face any challenges in the Python program learning assignment. Regarding implementation, the chatbot will be attached as an assistance facility in the form of a prompt where the user queries are passed on to the backend and

sent to the AI model in order to generate responses. Response received is then immediately shown in the application as immediate learning support.

In the development process, we tried two versions of chatbots. The original one was a direct input-output method and with no extra prompt restriction. In this implementation, the user only needed a small amount of instruction to send the message to the AI model, which then could provide a response of any type. This version worked, but the feedback was usually too long, inadvertently varied in style, and occasionally contained some information that was out-of-context regarding Python learning(Fig 6.1.4).

A second version was introduced through prompt engineering to enhance the quality of response and relevance to education. Here, a set query response was introduced before sending user query to the model. The role of the chatbot was clearly stated in the system prompt as a friendly and helpful Python programming tutor to beginners. It also defined some response limits, such as explaining Python concepts in straightforward language, giving examples where useful, answering questions about Python programming issues only, keeping answers brief but full, and employing motivating language, and referral of out-of-topic questions elsewhere to Python programming-related information. Technically, this prompt is an instruction layer to guide the behaviour of the AI model prior to generation. The backend allows the fixed system prompt to be combined with question asked by the user and passed to the model as input instead of just using the raw user input. This will enable the chatbot to have a more consistent teaching style and generate results more in line with the learning goals of the application(Fig 6.1.1 to 6.1.3). The prompt-based design, too, enhances the uniformity between the various user interactions, rendering the chatbot more pertinent, more formalized and more appropriate to amateur learners of Python(Fig 6.1.5).

6.2 Exercise Implementation

Figure 6.2.1 True/False Exercise screenshot

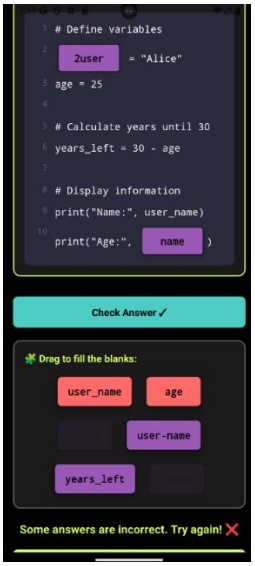


Figure 6.2.2 Drag-and-Drop Exercise screenshot

Figure 6.2.3 Manual Input Exercise screenshot

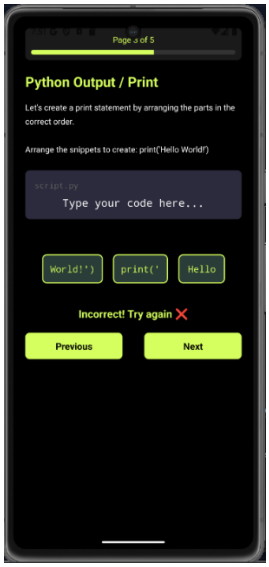


Figure 6.2.4 Sequencing Exercise screenshot

The system uses different forms of exercise(Fig 6.2.1 to 6.2.4) as one more possible way of diversifying the practice activities. All the formats are aimed at one particular part of learning Python. To provide more direct coding practice, there are coding exercises where the learner writes and executes Python code within the application, and fill-in-the-blank and reorganisation activities where the syntax, logic, and structure of a program are guided more closely. The drag-and-drop style gives the phenomenon of interactivity also because it requires the students to relocate the correct answers to the corresponding target positions or categories.

The exercise module is technically set up to dynamically load question data, and provides various interfaces based on the type of exercise. In the case of coding exercises, the learner writes code in Python and puts it in the backend to run, and the result or error notification is returned and immediately displayed. In fill-in-the-blank, reorganisation and drag-and-drop activities, the system will match the answers chosen or rearranged by a learner to the genuine answers kept in the system.

The interface in the drag-and-drop exercise (Fig 6.2.2) , in particular, consists of four major elements, namely, the display of the task, the movable answer options, the progress bar, and the answer validation part. The question or partially completed Python sample is displayed in the task display control with the choices of answers displayed as draggable labels which can be drag-and-dropped to the corresponding target positions

or categories. The system ensures that the learner clicks the relevant checking or navigation button after completing the activity and the system validates the answer and sends instant feedback. In case of wrong answer a learner is informed and given chance to retrial. This real-time verification process can be used to mark down and correct errors faster, and to foster experience learning in practice.

6.3 Simplify the error messages

```
File "C:\Users\user\Desktop\Code\learning app\python-backend\temp.py", line 1
    print('hello world'
          ^
SyntaxError: '(' was never closed
```

Figure 6.3.1 default Python error output

```
tracebackLines:
[0] "File \"C:\\Users\\user\\Desktop\\Code\\learning app\\python-backend\\temp.py\", line 1"
[1] "print(\"hello world\""
[2] "^"
[3] "SyntaxError: '(' was never closed"
```

Figure 6.3.2 Example of traceback lines after splitting

Figure 6.3.3 Simplify error message result

A downside of Python is that the error messages are usually long and hard to decode, particularly as a beginner. We even envisaged building a completely bespoke error-message module in the early project plan. In development, however, this was realized to be impractical within the time available to the project, since Python exceptions are quite varied and would need a lot of handling logic to exhaustively cover. Consequently, a less idealistic error-formatting strategy was embraced based on a backend.

During the implementation, the user executing the Python code captures both the original error message and the complete Python traceback produced as the Python code is executed. A system is not going to pass the entire traceback to the user, but instead it processes the traceback and passes it to the application. The traceback text is separated into the lines in the first step as illustrated (Figure 6.3.2). Individual lines are then narrowed down to eliminate the unnecessary space, and blank lines are filtered. When this step has finished, the backend then pulls out the ultimate useful traceback line,

which typically includes the most vital exception type and its corresponding message. This is then set as the short error text.

The brief message is then further accompanied by a remedial description, as incorporating a simple explanation of why the feedback is important and common beginner code errors, including syntax errors, indentation errors, undefined variables, and type errors. By doing so, the system minimises the technical detail the learner can see whilst still keeping the cause of failure. The processed response is then sent back to the application as JSON formatted which the frontend can read and show simplified and beginner friendly response immediately. Unlike the default Python traceback which includes lots of complex and technical data about debugging (Figure 6.3.1), the simplified feedback is less technical and basically only identifies the relevant issue and displays it in a more accessible and beginner-friendly format (Figure 6.3.3).

6.4 Level Navigation and Progress Trackers Implementation

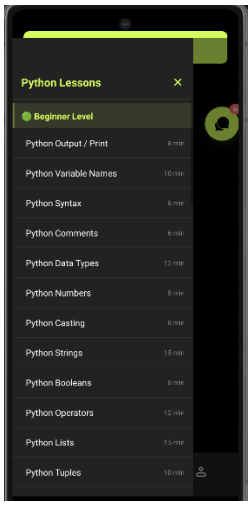


Figure 6.4.1
Beginner level in the lesson navigation drawer

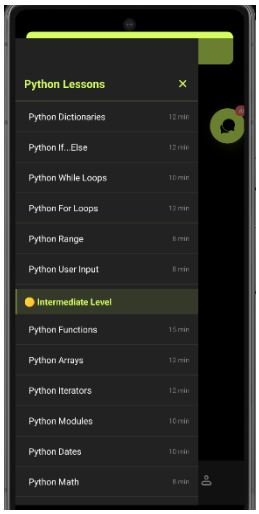


Figure 6.4.2
Intermediate level in the lesson navigation drawer

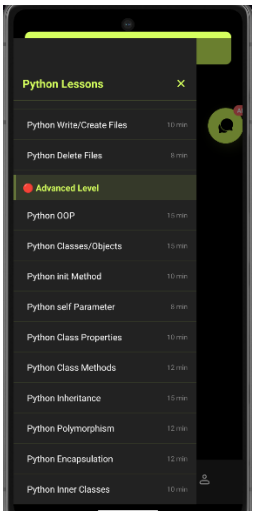


Figure 6.4.3
Advanced level in the lesson navigation drawer

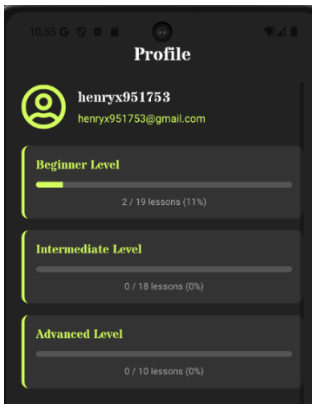


Figure 6.4.4 Progree tracker picture

This section includes implementing the level navigation feature in the implementation of the lesson. The application categorizes learning materials into three levels, which are: Beginner, Intermediate, and Advanced(Fig 6.4.1 to 6.4.2). A navigation bar is positioned at the top of the page instead of showing all the items of the lesson on the front page. As the user taps this bar, a left-side drawer is opened on which the entire list of lessons by level has been placed.

The functionality is achieved with the help of lesson records saved in Firebase Cloud Firestore. Attributes found in each lesson document are: title, duration and level. Once

the lesson data is loaded, the front end gathers the records based on the level value and displays them under the relative section on the drawer. This enables the exhibited lesson list to be driven by an interface instead of being challenging-coded. This design is aimed to allow users to access the sensation of navigating to the lesson category they would like to study anytime.

Besides level navigation, it also has progress indication, which is used by the learners to track their status regarding total learning status. Technically, every user has his or her own progress record, which is held in Firebase Cloud Firestore. This is a user-centered record where lessons and progress of the course in various levels are followed. The application will update the user record that has the progress data to show the corresponding progress in the learner after completing a lesson. This is then accessed and computed by the frontend to determine the completion status, e.g. the number of lessons completed and the percentage of progress. On the basis of such results, this system shows the progress information in the user interface where the learner can know the amount of content he has taken.

6.5 Interactive Interface Implementation

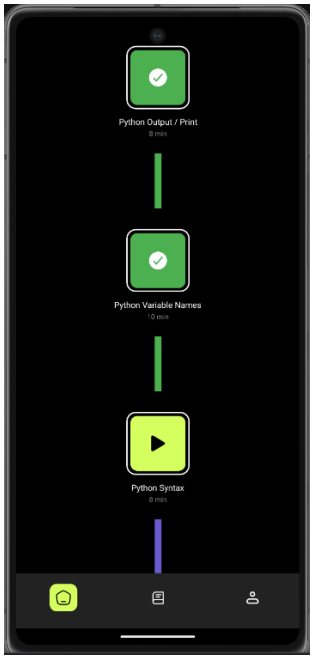


Figure 6.5.1 home page screenshot

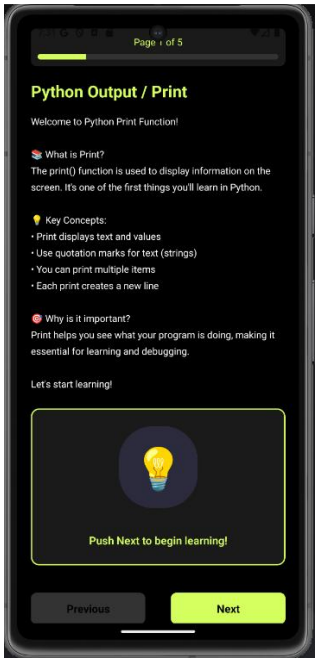


Figure 6.5.2 lesson page screenshot

Figure 6.5.3 Page-based progress display implemented in the exercise interface to show lesson completion flow.

Mobile learning application user interface was designed in a card-based format to enable easy and effective navigation via mobile gadgets. Lessons on the home page are

represented by individual interactive cards created based on lesson data in Cloud Firestore. Each card shows the main information like the name of the lesson and the time required to complete the lesson and one can easily find learning activities and learn without interacting a great deal.

All individual lesson cards are used as interactive UI items that are connected to individual lesson pages(Fig 6.5.1). Upon choice of a card, the application loads the learning material and other exercises covering that particular topic. This generates a systematic progression of learning where students transition between lesson choice to concept clarification and to subsequent tasks of follow-up learning(Fig 6.5.2). The links should be straight forward and simple in nature to allow the user to navigate to different tabs of the application such as home page, lesson content, exercises and the chatbot help desk in order to minimize the number of steps needed before reaching the required destination.

An in-lesson progress bar was provided in the exercise interface to guide users on the multi-step activity. This type of progress indicator shows the position that a learner is at in the activity sequence, i.e., Page 4 of 4, etc.(Fig 6.5.3), to the user so that they can quickly see the progress that they have made in a lesson. A dynamically updated progress measure is used to help the learner keep track of their position through page transitions, avoiding confusion in longer complete exercise sequences. This implementation helps to introduce a better step-by-step education experience and allows easier interaction among novice users.

Another use of the interface is that it uses a similar visual implementation in various pages. The primary theme is a dark background with the buttons, progress indicators, and interactive prompts being given fluorescent green accent colours. These components are re-purposed across the application to make it more visually consistent and emphasize the point of action. To enhance readability and contrast against the varying sizes of mobile screens, text contents are shown in white and light grey. The home page is currently being developed, so two alternative versions of the UI are being applied to compare the results. One version was an simplified card-based format, and the other provided textual descriptions of lesson content in more detail. These options were further utilized in the usability test to find out which home page design was more effective.

7. Evaluatation

7.1 Interface Usability Evaluation

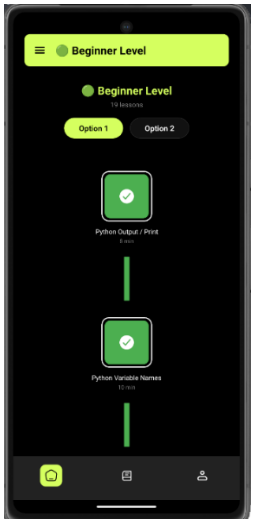


Figure 7.1.1: Option A – Simplified card-based lesson interface

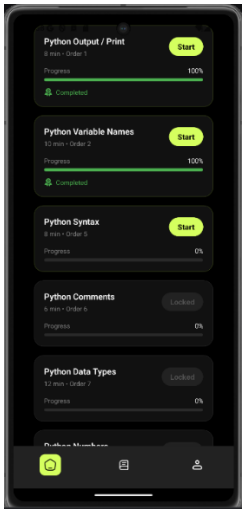


Figure 7.1.2: Option B – Text-rich interface with detailed descriptions

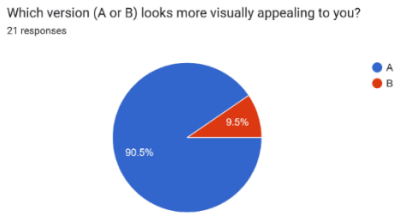


Figure 7.1.3: Questionnaire result

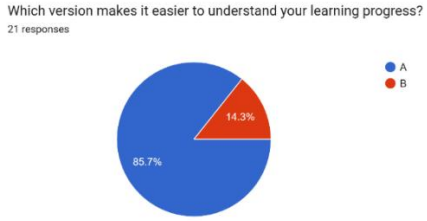


Figure 7.1.4: Questionnaire result

A questionnaire survey was conducted using Google Forms to evaluate the usability of the homepage interface design . Participant were show two interface designs: Option A (Fig. 7.1.1), which used simplified card-based layout, and Option B (Fig. 7.1.2), which provid more detailed textual descriptions of the lesson content. The questionnaire asked participants to compare the two design in terms of clarity of information, ease of understanding, visual simplicity, and overall preference.

The results of the questionnaire show that most participants prefer Option A over Option B. As shown in Figs. 7.1.3 and 7.1.4, respondents found Option A clearer, simpler, and easier to use. The simplified card-based interface enabled participants to access lessons more easily and helped them remember the next learning task. In contrast, although Option B provide more detailed information, some participants felt that the large amount of text reduced their engagement, especially when using mobile device. These finding suggest that simpler visual design improve usability by reducing cognitive load and allowing user to navigate the learning materials more efficiently. Therefore, Option A is selected as the final interface design for the application.

7.2 Error message evaluation

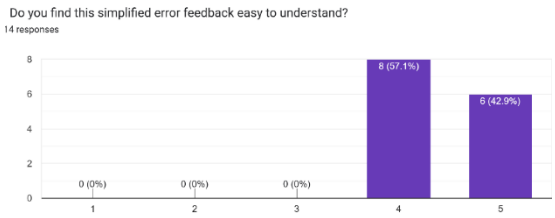


Figure 7.2.1: Questionnaire result

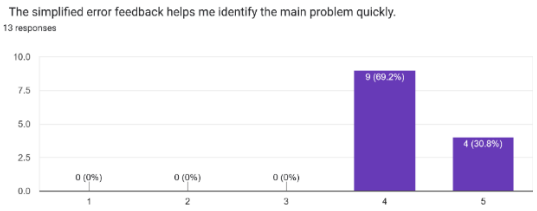


Figure 7.2.2: Questionnaire result

The simplified error feedback option was positively accepted by the subjects. On the question whether the simplified error feedback was easy to understand, 57.1% of respondents rated it 4, 42.9% rated it 5 (Fig 7.2.1). No one chose between 1 and 3 which means that none of the participants found simplified feedback unclear and unintelligible. Likewise, the answers to the question of whether the simplified error feedback allowed users to quickly determine the key issue were also overwhelmingly positive with 69.2% rating 4 and 30.8% rating 5 (Fig 7.2.2). Once again, the rating did not show any lows, indicating that the feature worked in assisting the user in identifying the major problem in their code more efficiently.

Generally, the results indicate that the simplified error feedback was effective in enhancing the usability of the learning system. The system no longer showed raw Python error messages, but instead included an easier to understand feedback that could be interpreted more quickly by the user to diagnose the issue. This is particularly true with novice learners who might not understand standard programming error messages.

7.3 Prompt-based chatbot vs unconstrained chatbot evaluation

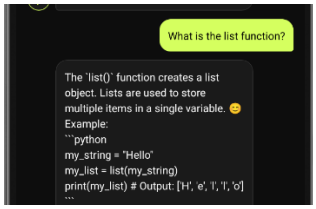


Figure 7.3.1 Option-A Prompt-based chatbot

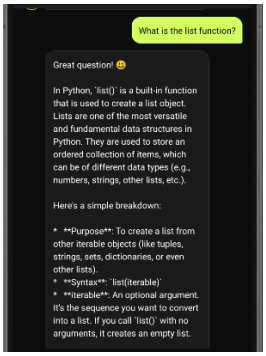


Figure 7.3.2 Option-B unconstrained chatbot

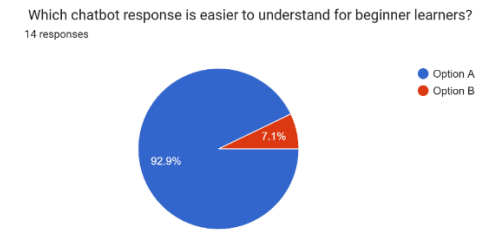


Figure 7.3.3: Questionnaire result

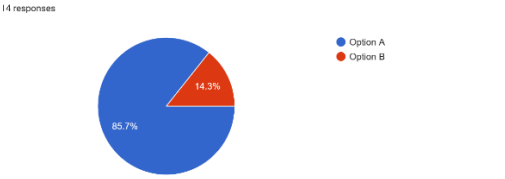


Figure 7.3.4: Questionnaire result

To test the efficiency of the chatbot design, two versions of a chatbot were compared, one version where the chatbot did not have constraint prompts(Fig 7.3.2) and the other version where the chatbot used a prompt based instruction to direct the final AI responses(Fig 7.3.1). In the original, the chatbot reacts to any question asked by a user freely. Although the answers are mostly true, the descriptions are lengthy and are often full of superfluous details. Also, the chatbot can provide answers to questions that are not relevant to Python programmation, which makes it less useful as a learning assistant. The second version relies on the system prompt to limit the responses delivered by the chatbot to questions related to Python and deliver brief explanations.

According to the comparison findings, the prompt-based chatbot will generate more expert and specific replies. The instructions are shorter and more in line with the educational aspect of the usage. In Figure 7.3.3 and 7.3.4, differences between two versions of chatbots are presented. Because the presentation can be seen in the comparison, the quick-chatbot offers more specific and immediate answers, which is why it will be better to use with novice learners when using the application.

7.4 Level Navigation and Progress Trackers evaluation

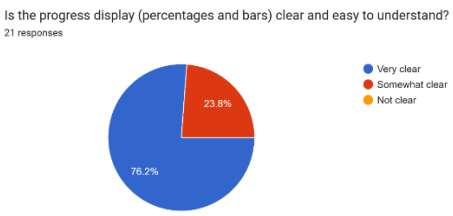


Figure 7.4.1 Questionnaire result

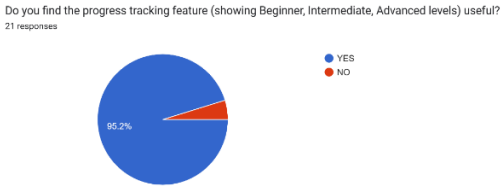


Figure 7.4.2 Questionnaire result

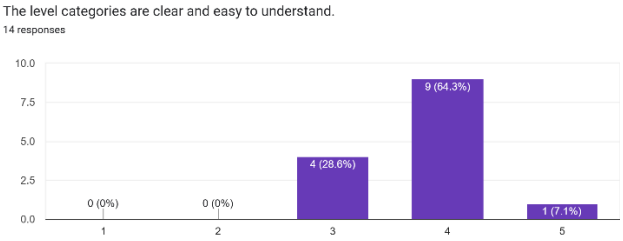


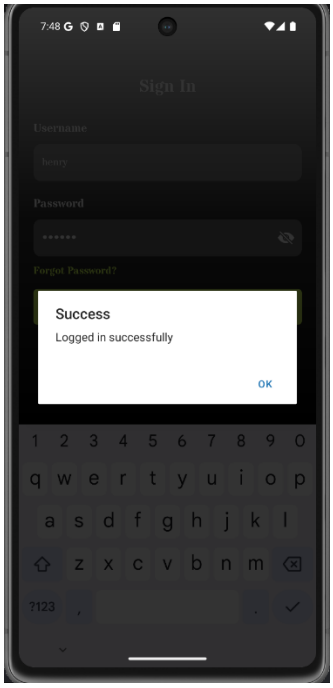
Figure 7.4.2 Questionnaire result

The progress tracker feature was highly rated by the ratees as well. When asked about the clarity of the display of progress, 76.2 percent of prospective customers rated the percentages and progress bars as being very clear, with 23.8 percent rating them as somewhat clear (Fig 7.4.1). None of the participants chose not clear, which means that the visual representation of the progress was not very difficult to follow as a rule.

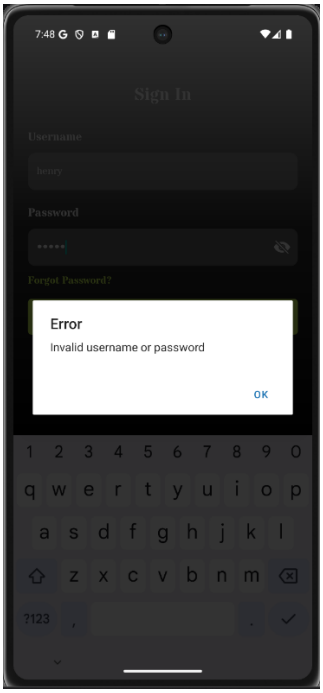
Also, the progress tracking option was rated as being helpful with 95.2% of respondents indicating that displaying Beginner, Intermediate, and Advanced levels were helpful and only 4.8% indicated the opposite(Fig 7.4.2). This can indicate that the feature was effective in enabling users to make sense of their learning progress and status in the app.

The respondents also rated the level categories positively. In the response to the statement, The level categories are clear and easy to understand, the average level of response was 3.79 out of 5, and the reasons presented were 64.3% 4, 28.6% 3, and 7.1% 5 and 0% 1 and 2(Fig 7.4.3). These findings indicate that the level-based structure was usually considered to be clear and understandable.

7.5 Test case



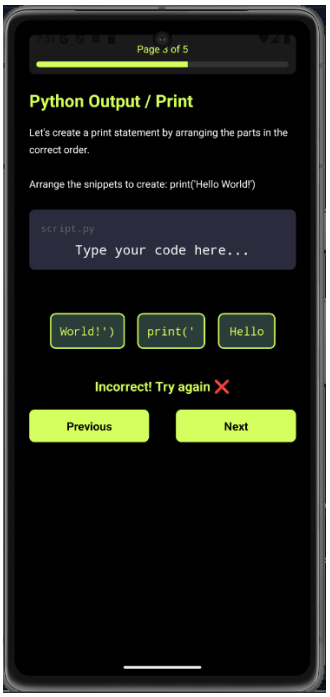
Figures 9.1 Successful Login case



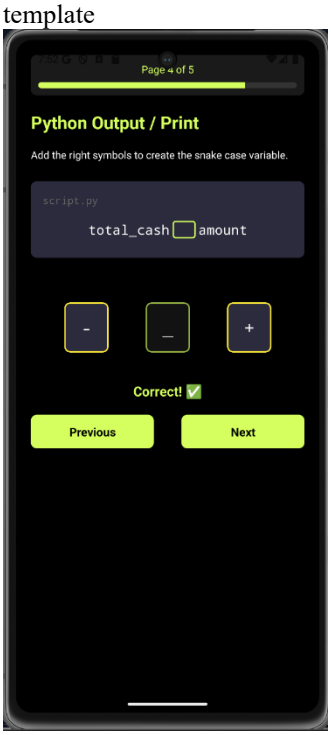
Figures 9.2 Fail Login case

Figures 9.3 Successful output shown after selecting a sample template

Figures 9.4 Reorganize the sentence success case



Figures 9.5 Reorganize the sentence fail case



Figures 9.6 Fill in the blanks success case

Figures 9.7 Successful run of user-entered Python code

Figure 9.8 Example of an easy-to-understand error message

Figure 9.9 Successful input correct answer case

Figure 9.10 Classification exercise: error message displayed for incorrect categorisation

Figure 9.11 Correct answer case in drag-and-drop exercis

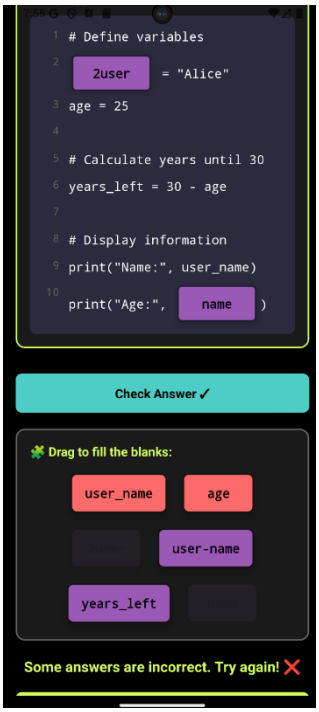


Figure 9.12 Wrong answer case in drag-and-drop exercis

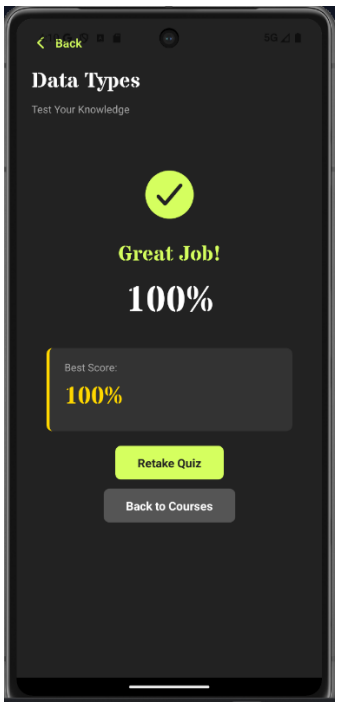


Figure 9.13 Successful Quiz Completion with F

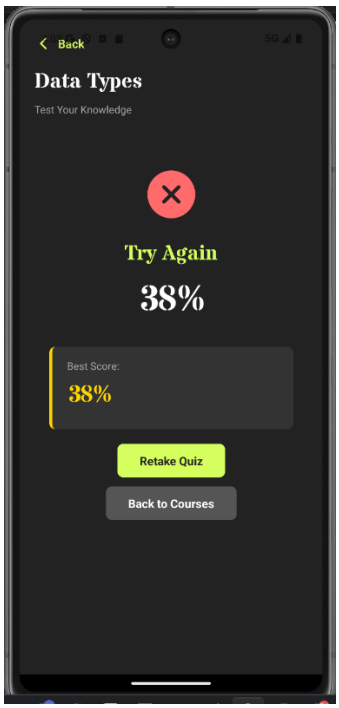


Figure 9.14 Quiz Feedback for Incorrect Answers

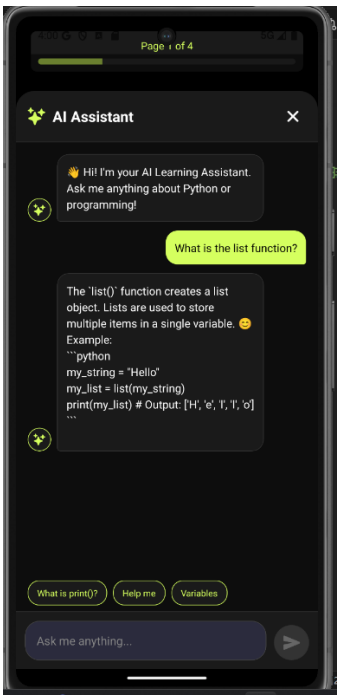


Figure 9.15 AI Chatbot Answering a User Question

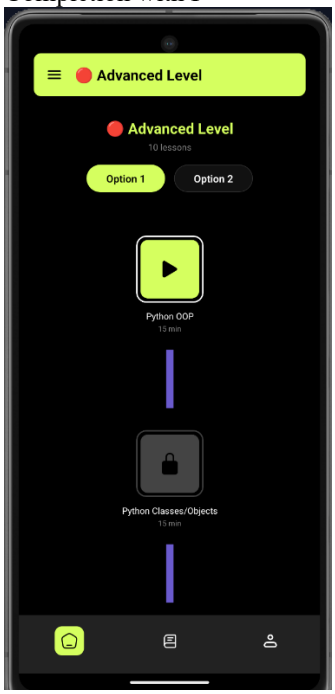


Figure 9.16 Navigation to Advanced Level

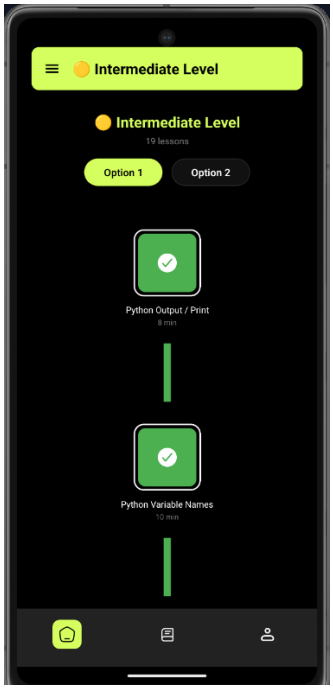


Figure 9.17 Navigation to Intermediate Level

TC ID	Feature / Module	Test Scenario	Preconditions	Steps / Input	Expected Result	Actual Result	Status
TC-01	Authentication	Successful login	User account exists	Enter valid name/password → click Login	Login succeeds and user enters main page	Fig. 9.1	Pass

TC-02	Authentication	Failed login	None	Enter invalid credentials → click Login	Error message shown; user remains on login page	Fig. 9.2	Pass
TC-03	Tap template	Tap sample template shows output	Lesson page accessible	Tap a template code snippet	Output panel displays result correctly	Fig. 9.3	Pass
TC-04	Reorganise sentence	Correct answer case	Exercise loaded	Reorganise sentence correctly → Check	Correct feedback shown	Fig. 9.4	Pass
TC-05	Reorganise sentence	Wrong answer case	Exercise loaded	Reorganise sentence incorrectly → Check	Incorrect feedback shown	Fig. 9.5	Pass
TC-06	Fill in the blanks	Correct answer case	Exercise loaded	Fill blanks with correct items → Check	Correct feedback shown	Fig. 9.6	Pass
TC-07	Code execution	Successful run of user code	Backend running	Input valid Python code → Run	stdout shown; execution successful	Fig. 9.7	Pass
TC-08	Code execution + error handling	Simplified error message case	Backend running	Input code with error → Run	Beginner-friendly error message displayed	Fig. 9.8	Pass
TC-09	Drag-and-drop	Correct answer case	Exercise loaded	Drag to correct positions → Check	Correct feedback shown	Fig. 9.9	Pass
TC-10	Classification	Incorrect categorisation case	Exercise loaded	Sort items incorrectly → Check	“Incorrect ” message shown	Fig. 9.10	Pass
TC-11	Drag-and-drop	Correct answer case	Exercise loaded	Drag items wrongly → Check	Correct feedback shown	Fig. 9.11	Pass
TC-12	Drag-and-drop	Wrong answer case	Exercise loaded	Drag items wrongly → Check	Retry prompt shown	Fig. 9.12	Pass
TC-13	Quiz	Quiz can achieve full marks	User has entered a quiz page	Answer all quiz questions correctly → submit quiz	System shows full score of 100%	Fig 9.13	Pass
TC-14	Quiz Review	User can review	User has completed a	Answer some quiz questions	Incorrect score and	Fig 9.14	Pass

		incorrect quiz result	quiz with wrong answers	incorrectly → submit quiz	retry option shown		
TC-15	AI Chatbot	Chatbot answers user question	User is on lesson page and chatbot is accessible	Open AI Assistant → enter a Python-related question	Chatbot returns a relevant	Fig 9.15	Pass
TC-16	Level Navigation	Switch to Advanced level	User is logged in and level page is accessible	Select / navigate to Advanced Level	displays the Advanced Level page and its lesson path	Fig 9.16	Pass
TC-17	Level Navigation	Switch to Intermediate level	User is logged in and level page is accessible	User is logged in and level page is accessible	displays the Intermediate Level page and its lesson path	Fig 9.17	Pass

Table 9.1 Test Cases

After implementing the main functions, we conducteds functional testing to verify key user flows, including login, interactive exercises, code execution, and error feedback. Each test case are recorded in Table 9.1 and screenshots are provided as evidence (Figures 9.1-9.17), All listed test cases passed based on the expected results.

8. Conclusion

In this project, the Python based mobile based learning application has been created to help beginner learners go through a more organized and interesting learning process. Final features consist of level-based lessons, interactive exercises, code execution, simplified error feedback, progress tracking and an AI-based chatbot to help with learning. These characteristics were created to overcome various issues prevalent in current platforms, such as lack of variety in practice, vague progression, and lack of immediate assistance to novices.

The results of the evaluation indicate that the features developed were mostly easy to understand and helpful to the users, especially the simplified error feedback, chatbot design, and tracker. Altogether, the project demonstrates that mobile-first and beginner-friendly approach could enhance the ease of learners accessing and using Python. Nevertheless, there remains a content and evaluation scale limitation with the present system. The next task should be to increase the content of the lessons, stabilize the system, and perform a greater number of user tests to prove the efficiency of the application once more.

9. References

[1] Chan, J., & Chan, J. (2024, August 23). Why learning programming still matters in the era of AI. AI Innovation Academy. https://ai-innovation-academy.com/programming-in-the-ai-era/

[2] Patil, H., Mahandule, V., & Gunjal, A. (2025). Python in the Evolution of AI: A Comparative Study of Emerging Technologies. SSRN Electronic Journal. https://doi.org/10.2139/ssrn.5075929

[3] Idowu, S., Sens, Y., Berger, T., Krueger, J., & Vierhauser, M. (2024). A Large-Scale Study of ML-Related Python Projects. S. Idowu, Y. Sens, T. Berger, J. Krüger, and M. Vierhauser, 1272–1281. https://doi.org/10.1145/3605098.3636056

[4] DASCA. (n.d.). What Makes Python the go-to Language for Data Scientists? https://www.dasca.org/world-of-data-science/article/what-makes-python-the-go-to-language-for-data-scientists

[5] Garzón, J., Burgos, D., Kinshuk, & Tlili, A. (2025). Mobile learning significantly enhances student learning gains: A meta-analysis and research synthesis. Computers & Education, 238, 105415. https://doi.org/10.1016/j.compedu.2025.105415

[6] Traxler, J., & Kukulska-Hulme, A. (2015b). Mobile learning. https://doi.org/10.4324/9780203076095

[7] Corbeil, J. R., Khan, B. H., & Corbeil, M. E. (2021b). Microlearning in the digital age. https://doi.org/10.4324/9780367821623

[8] Chen, A., Wei, Y., Le, H., & Zhang, Y. (2025b). Learning by teaching with ChatGPT : The effect of teachable ChatGPT agent on programming education. British Journal of Educational Technology, 57(1), 163–184. https://doi.org/10.1111/bjet.70001

[9] Schliep, P. A. (n.d.-b). Usability of error messages for introductory students. University of Minnesota Morris Digital Well. https://digitalcommons.morris.umn.edu/horizons/vol2/iss2/5/

[10] Bergdahl, N. (2022). Engagement and disengagement in online learning. Computers & Education, 188, 104561. https://doi.org/10.1016/j.compedu.2022.104561

[11] Dirzyte, A., Perminas, A., Kaminskis, L., Žebrauskas, G., Pačiauskienė, Ž. S. –., Šliogerienė, J., Suchanova, J., Knabikienė, R. R. –., Patapas, A., & Gajdosikiene, I. (2023). Factors contributing to dropping out of adults' programming e-learning. Heliyon, 9(12), e22113. https://doi.org/10.1016/j.heliyon.2023.e22113

[12] Van Der Kleij, F. M., Feskens, R. C. W., & Eggen, T. J. H. M. (2015). Effects of feedback in a Computer-Based Learning environment on students' learning outcomes. Review of Educational Research, 85(4), 475–511. https://doi.org/10.3102/0034654314564881

[13] Curum, B., & Khedo, K. K. (2020c). Cognitive load management in mobile learning systems: principles and theories. Journal of Computers in Education, 8(1), 109–136. https://doi.org/10.1007/s40692-020-00173-6

[14] Zainal, N. F. A., Shahrani, S., Yatim, N. F. M., Rahman, R. A., Rahmat, M., & Latih, R. (2012). Students' perception and motivation towards programming. Procedia - Social and Behavioral Sciences, 59, 277–286. https://doi.org/10.1016/j.sbspro.2012.09.276